Wanderlust Tour Management Ecosystem

Full-Stack Spring Boot 3 + Java 21 Backend & System Design Interview Master Guide

Developer: Ajay Kushwaha **Academic Degree:** Bachelor of Engineering (IT), Mumbai University

Frameworks: Spring Boot 3.1.4 • Java 21 • Hibernate 6 • QueryDSL **Database:** H2 (Dev) / PostgreSQL (Prod) via HikariCP

Deployment: Render Cloud PaaS (Dockerized JRE 21)

This guide covers technical questions, database design, transactions, concurrency, performance optimizations, and future scalability scenarios tailored specifically to the Wanderlust Tour Management backend codebase.

1. High-Level Architecture & Request Lifecycle

Q1: What architectural pattern does this backend follow? How does a request travel through the system?

Intent: Verifies understanding of N-Tier Layered Architecture and decoupling.

This project follows the industry-standard **N-Tier Layered Architecture** with strict separation of concerns:

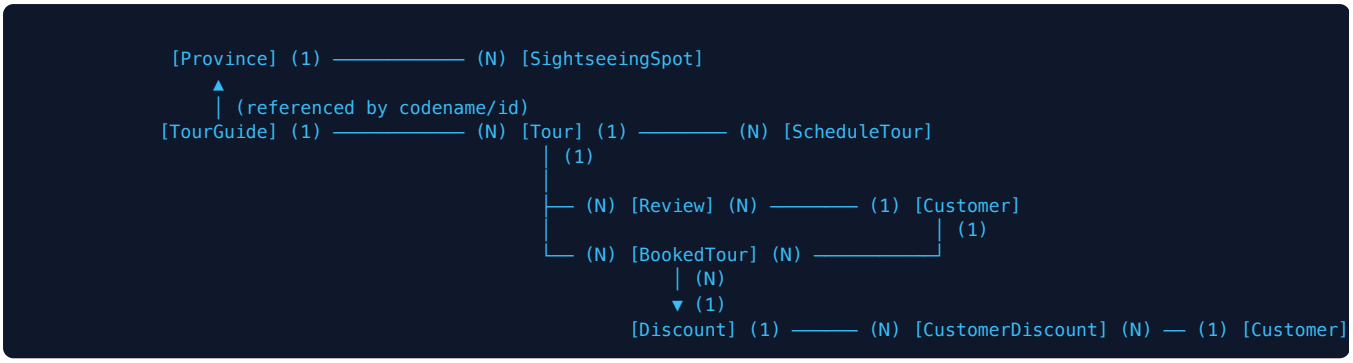
```
[React Frontend] (HTTP REST / Axios with JWT Bearer)
       |
       ▼
[Security & Filter Layer] (CorsConfigurationSource ──► JwtAuthenticationFilter)
       |
       ▼
[Presentation / Controller Layer] (@RestController ──► @Valid RequestDTOs ──► GlobalExceptionHandler)
       |
       ▼
[Business Service Layer] (@Service ──► Business Validation ──► Transactions @Transactional)
       |
       ▼
[Data Access / DAO Layer] (Spring Data JpaRepository ──► Custom QueryDSL BookedTourDaoImpl)
       |
       ▼
[ORM / Entity Layer] (Hibernate 6 Entities ──► HikariCP Connection Pool)
       |
       ▼
[Database] (H2 Engine / PostgreSQL on Render)
```

EXACT RESPONSE TO INTERVIEWER:

"We implemented an N-Tier architecture consisting of Controller, DTO, Service, DAO/Repository, and Entity layers. A request passes through the Spring Security Filter Chain for stateless JWT verification, reaches the @RestController which validates inputs via Jakarta Validation DTOs, enters the @Service layer where business rules and transactions are executed, queries the database via Spring Data JPA and QueryDSL, and returns clean ResponseDTOs back to the client to avoid exposing database internals."

2. Database Schema, Tables & Entity Relationships

The schema consists of **11 relational entities** structured around core tourism operations:



Q2: How did you model the Many-to-Many relationship between Customer and Discount? Why not use standard @ManyToMany?

Intent: Tests deep JPA mapping skills and enterprise data modeling best practices.

In `CustomerDiscount.java`, we explicitly avoided bare `@ManyToMany` and created a dedicated intermediate entity.

Why bare @ManyToMany is an antipattern: Bare `@ManyToMany` generates a hidden join table that cannot store extra metadata. In real-world travel systems, you must record relationship attributes such as coupon redemption status (`ACTIVE` , `EXPIRED` , `USED`) and claimed timestamp.

EXACT RESPONSE TO INTERVIEWER:

"Rather than using standard @ManyToMany, we modeled the relationship through a dedicated join entity: CustomerDiscount, using two @ManyToOne associations (@ManyToOne Customer and @ManyToOne Discount). This allows us to track redemption state, usage count, and lifecycle timestamps directly on the link."

Q3: What FetchType strategies did you adopt across @OneToMany and @ManyToOne, and why?

Intent: Tests knowledge of Hibernate memory management, lazy loading, and LazyInitializationException.

By default in JPA, `@OneToMany` defaults to **LAZY**, while `@ManyToOne` defaults to **EAGER**.

Design Decision: For heavy collections (such as `Tour.bookedTours` , `Tour.reviews` , and `Customer.bookedTours`), we strictly maintain **LAZY fetching**. When a user browses the tour catalog, the system should not pull hundreds of customer booking records into JVM heap memory. Related records are only fetched when explicitly requested or joined in the service layer.

3. Database Connectivity, HikariCP & Configuration

Q4: How does Spring Boot manage database connections under the hood? What connection pool is used?

Intent: Tests understanding of JDBC connection pooling and runtime configuration.

Spring Boot automatically configures **HikariCP** (the fastest lightweight JDBC connection pool). When the application boots, Hikari creates a pool of pre-established physical connections (`HikariPool-1`).

In our `application.yml` :

```
spring:
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:h2:file:./data/tour_management;NON_KEYWORDS=DAY}
    driver-class-name: ${SPRING_DATASOURCE_DRIVER_CLASS_NAME:org.h2.Driver}
    username: ${SPRING_DATASOURCE_USERNAME:sa}
    password: ${SPRING_DATASOURCE_PASSWORD:}
```

EXACT RESPONSE TO INTERVIEWER:

"Spring Boot uses HikariCP for connection pooling. In local and college testing, we use a file-backed H2 embedded database (jdbc:h2:file:./data/tour_management) which persists data across restarts. For production deployment on Render, we use environment variable overrides (SPRING_DATASOURCE_URL, USERNAME, PASSWORD) to connect to PostgreSQL without changing a single line of Java code."

Q5: Why did you configure spring.jpa.open-in-view: false in application.yml?

Intent: High-level architectural question. Open Session In View (OSIV) traps many developers.

By default, Spring Boot sets `open-in-view: true` , holding the database connection open for the entire HTTP request lifecycle, including during Jackson JSON serialization in the controller.

Why we explicitly set it to false:

1. **Prevents Connection Starvation:** Under high concurrent traffic, holding connections while transmitting HTTP bytes exhausts the Hikari pool. Setting it to `false` releases the connection as soon as the `@Service` transaction finishes.
2. **Enforces Clean Architecture:** Prevents accidental lazy-loading queries during serialization in the presentation layer.

Q6: What is ddl-auto: update? Would you use it in a banking or production enterprise app?

Intent: Distinguishes student projects from enterprise production knowledge.

`hibernate.ddl-auto: update` inspects Entity class annotations on startup and compares them to the DB schema, adding missing tables or columns without dropping existing rows.

EXACT RESPONSE TO INTERVIEWER:

"While ddl-auto: update is convenient for development and college evaluation, we would NEVER use it in strict production. It cannot safely handle column renames, drops, or index optimizations and can cause unpredictable table locks. In production, we set ddl-auto: validate (or none) and use version-controlled database migration tools like Flyway or Liquibase."

4. Real-World Scenario-Based Technical Questions

Q7: Scenario: Two customers attempt to book the last available seat on the same tour at the exact same millisecond. How do you prevent overbooking?

Intent: Tests concurrency, race conditions, ACID properties, and database locking.

This is a classic **concurrency race condition**. If two threads read `availableSeats = 1` simultaneously, both pass the check and decrement, resulting in `availableSeats = -1` (Overbooking).

Solution 1: Optimistic Locking (@Version) — Recommended for High Read web apps:

```
@Entity
public class Tour {
    @Version
    private Long version;
    // ...
}
```

Hibernate checks the version column upon commit. The first user commits successfully (version becomes 2). The second user's commit detects a version mismatch and throws `OptimisticLockException`. Our `GlobalExceptionHandler` catches this and responds: *"Seat already reserved by another user. Please choose another tour."*

Solution 2: Pessimistic Locking (SELECT ... FOR UPDATE):

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT t FROM Tour t WHERE t.id = :id")
Optional<Tour> findByIdWithLock(@Param("id") Long id);
```

Locks the database row exclusively, forcing thread 2 to wait until thread 1 commits or rolls back.

Q8: Scenario: The booking record is saved in the database, but sending the confirmation/reminder email fails due to an SMTP timeout. Does the booking rollback?

Intent: Tests transaction boundaries (@Transactional) and decoupling of non-critical services.

If the email method is synchronous inside the `@Transactional` method and throws an unhandled `RuntimeException`, Spring's transaction manager will trigger a **rollback**, erasing the booking.

How our architecture handles this: In our system, email dispatch is considered a **non-critical side effect**. We decouple email sending in a `try-catch` block or via an asynchronous service (`/request-payment`). The booking is persisted with status `PENDING`, ensuring that a transient SMTP network glitch never aborts a customer's confirmed reservation.

Q9: What is the N+1 Query Problem in JPA, and how does your project prevent it?

Intent: The single most common ORM performance question asked in interviews.

The Problem: Querying 10 tours with `tourRepository.findAll()` executes 1 SQL query. But if you subsequently access each tour's guide (`tour.getGuide().getName()`), Hibernate executes 10 additional queries—resulting in **1 + N (11) total queries**.

Three Ways to Fix It:

1. **JOIN FETCH in JPQL:** `@Query("SELECT t FROM Tour t LEFT JOIN FETCH t.guide LEFT JOIN FETCH t.scheduleTours")` executes a single SQL JOIN.
2. **@EntityGraph:** Declaratively specify the entity attributes that should be fetched eagerly in one query.
3. **DTO Projections:** Query only the required columns directly into a flat DTO constructor.

5. Spring Boot Core, Security & DTO Design

Q10: Why did you use the DTO (Data Transfer Object) pattern instead of exposing Entities directly in REST Controllers?

Intent: Tests understanding of security, serialization, and architectural decoupling.

Exposing JPA entities directly in REST controllers introduces 3 critical flaws:

Issue	What happens without DTOs	How DTOs solve it
Infinite JSON Recursion	Bidirectional JPA relationships (`Tour <-> BookedTour`) trigger Jackson infinite loops resulting in `StackOverflowError` and server crash.	Response DTOs extract only flat fields (id, name, price) without circular object references.
Mass-Assignment Vulnerability	An attacker can send `{"role": "ADMIN"}` or manipulate prices during registration or booking.	Request DTOs strictly whitelist permitted input fields.
Multipart File Uploads	A JPA entity cannot hold binary `MultipartFile` HTTP uploads (only DB column strings/URLs).	`CustomerReqDTO` accepts binary avatar/image streams, which the service uploads and converts into URL strings.

Q11: Why did you use QueryDSL in BookedTourImpl instead of standard Spring Data JPA method names?

Intent: Explains why custom DAO implementation exists alongside JpaRepository.

In `BookedTourImpl.java` , the Admin Dashboard requires **dynamic multi-criteria filtering** (optional start date, end date, booking status, and pagination):

```
JPAQuery<BookedTour> query = new JPAQuery<>(_entityManager);
QBookedTour bookedTour = QBookedTour.bookedTour;
query = query.from(bookedTour);

if (filter.getStartDate() != null && filter.getEndDate() != null) {
    query = query.where(bookedTour.bookingDate.between(filter.getStartDate(), endDate));
}
if (filter.getStatus() != null) {
    query = query.where(bookedTour.status.eq(EnumStatusBookedTour.fromString(filter.getStatus())));
}
return query.fetch();
```

EXACT RESPONSE TO INTERVIEWER:

"Spring Data JPA method derivation works well for static queries, but for dynamic multi-criteria dashboards where filters are optional, method names explode combinatorially. QueryDSL allows us to construct compile-time type-safe dynamic queries using generated Q-classes without messy string-concatenated SQL."

Q12: How does stateless JWT Authentication work in this project? How are passwords secured?

Intent: Tests Spring Security 6 architecture, filter chain, and cryptographic hashing.

1. **BCrypt Hashing:** Passwords in `Account` are never stored in plain text. They are hashed using `BCryptPasswordEncoder` with an automatic salt and 10+ work factor rounds, making them immune to rainbow table attacks.
2. **Authentication Filter:** We configured a custom `JwtAuthenticationFilter` registered before `UsernamePasswordAuthenticationFilter` in `SecurityFilterChain` .
3. **Stateless Validation:** The filter extracts the `Authorization: Bearer <token>` header, verifies the HMAC-SHA256 digital signature, and sets user details into `SecurityContextHolder` without querying the DB on every request.

6. "What Can We Add To...?" — Scalability & Production Roadmap

Interviewers frequently ask what you would add to scale the application to hundreds of thousands of users:

Q13: What can we add to reduce database load and make tour searches instantaneous?

We can add Redis Distributed In-Memory Caching (@Cacheable):
Tour details and destination highlights are read 95% of the time and modified only 5% of the time. By caching them in Redis with a 1-hour TTL, search response times drop from 150ms to **under 5ms**, and database read operations are reduced by over 80%. When an admin modifies a tour, `@CacheEvict` invalidates the cache.

Q14: Currently images are saved to the server's local disk. What can we add to make this cloud-native?

We can add AWS S3 (or Cloudinary) with CloudFront CDN and Presigned URLs:
Storing images on local disk fails when running multiple clustered server instances behind a load balancer. We can generate short-lived **S3 Presigned URLs** so the React frontend uploads photos directly to AWS S3, saving server network bandwidth and serving images via global edge CDNs.

Q15: What can we add to handle flash-sale booking spikes without server crashes?

We can add an Asynchronous Message Queue (Apache Kafka or RabbitMQ):
Instead of processing heavy seat deductions and invoice generation synchronously, the controller publishes a `BookingEvent` to Kafka and immediately returns `HTTP 202 Accepted`. Background consumers process the queue at a sustainable rate, eliminating thread starvation.

Q16: What can we add to convert mock bookings into real financial transactions?

We can add Payment Gateway Integration (Razorpay / Stripe) with Webhooks and Idempotency:
The backend initiates a payment order with Razorpay, the client pays via UPI/Cards, and Razorpay triggers a cryptographically signed webhook (`payment.captured`). We implement **Idempotency Keys** to prevent double charging on network retries.

Q17: What can we add to prevent API abuse, DDoS attacks, and brute-force logins?

We can add Rate Limiting via Bucket4j and Token Bucket Algorithm:
Throttle requests (e.g., maximum 60 requests/minute per IP address). On `/auth/login` , block an IP after 5 failed attempts. When a user logs out, store their JWT in a Redis token blacklist until expiration.

Q18: What can we add to monitor server health, JVM memory leaks, and slow SQL queries in production?

We can add Spring Boot Actuator + Prometheus + Grafana:
Actuator exposes JVM metrics, Prometheus scrapes them periodically, and Grafana renders dashboards monitoring active Hikari connection pool usage, heap memory, and HTTP response latencies.

7. Rapid-Fire Screening Cheat Sheet (20-Second Answers)

Topic / Question	Your 20-Second Answer
IoC & Dependency Injection	IoC inverts object creation from classes to the Spring container. DI injects dependencies via constructors rather than manual `new` keywords.
Constructor vs Field Injection	Constructor injection enforces immutability with `final` fields and allows easy unit testing without loading the full Spring context.
Bean Scope	Default is **Singleton** (1 instance per container). Others are Prototype, Request, and Session.
@RestController vs @Controller	`@RestController` is `@Controller` + `@ResponseBody`, returning JSON directly instead of rendering an HTML template.
save() vs saveAndFlush()	`save()` waits for the Hibernate session flush. `saveAndFlush()` immediately executes the SQL INSERT/UPDATE statement in the database.
Pagination Mechanism	Spring Data `PageRequest.of(page, size)` automatically appends `LIMIT` and `OFFSET` clauses to the generated SQL query.
Global Error Handling	Centralized via `@ControllerAdvice` and `@ExceptionHandler`, returning uniform `ErrorResponse` DTOs with standardized HTTP status codes.
CORS Handling	Configured in `SpringSecurity.java` with `CorsConfigurationSource` allowing credentials, headers, and HTTP methods across domains.
Packaging & Run	Built into an executable Fat JAR via `spring-boot-maven-plugin` and run via `java -jar app.jar` with embedded Tomcat.

💡 **Tip for your College Viva & Placement Interviews:**
Always introduce your project using this crisp pitch: *"Wanderlust is a full-stack, enterprise-grade travel and tour management platform. The backend is built with Spring Boot 3 and Java 21, following an N-tier layered architecture with Spring Data JPA, Hibernate ORM, and QueryDSL for dynamic financial analytics. It features stateless JWT authentication, HikariCP connection pooling, and centralized error handling, and is deployed live on Render via multi-stage Docker containers."*